

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Báo cáo thực hành cá nhân 1

Đề tài: Cài đặt một số tác vụ xử lý ảnh

Môn học: Thị giác máy tính

Sinh viên thực hiện:

Võ Ngọc Hiếu (23122027)

Giáo viên hướng dẫn:

TS. Võ Hoài Việt

ThS. Phạm Minh Hoàng

Ngày 1 tháng 4 năm 2026



Mục lục

1	Giới thiệu	1
2	Cấu trúc thư mục	1
3	Hướng dẫn cài đặt và sử dụng	2
3.1	Yêu cầu hệ thống và Cài đặt	2
3.2	Cấu hình môi trường Visual Studio Code	3
4	Cơ sở lý thuyết của các thuật toán xử lý ảnh	3
4.1	Phép tích chập 2D (2D Convolution) và Xử lý biên (Padding)	4
4.2	Chuyển đổi ảnh màu sang ảnh xám	4
4.3	Thay đổi độ sáng (Brightness)	5
4.4	Thay đổi độ tương phản (Contrast)	6
4.5	Bộ lọc trung bình (Average Filter)	7
4.6	Bộ lọc trung vị (Median Filter)	8
4.7	Bộ lọc Gauss (Gaussian Filter)	9
4.8	Phát hiện biên cạnh bằng Sobel kernel (Sobel Edge)	10
4.9	Phát hiện biên cạnh bằng Laplace kernel (Laplace Edge)	11

Danh sách hình vẽ

1	Ảnh ví dụ (Nguồn: wikipedia.org)	3
2	Kết quả chuyển đổi ảnh màu sang ảnh xám	5
3	Ảnh sau khi tăng độ sáng lên 50	6
4	Ảnh sau khi tăng độ tương phản 1.5 lần	7
5	Ảnh sau khi qua bộ lọc trung bình ($k=5$)	8
6	Ảnh sau khi qua bộ lọc trung vị ($k=5$)	9
7	Ảnh sau khi qua bộ lọc Gauss ($k=5$)	10
8	Kết quả phát hiện biên Sobel	11
9	Kết quả phát hiện biên Laplace	12

1 Giới thiệu

Báo cáo này trình bày cơ bản về chương trình, cách cài đặt và thuật toán xử lý ảnh được dùng trong chương trình. Chương trình này được lập trình bằng ngôn ngữ **C/C++** với thư viện **OpenCV**. Môi trường sử dụng để lập trình là **Arch Linux** với công cụ **Visual Studio Code**. Các chức năng xử lý ảnh của chương trình được cài đặt từ đầu, bao gồm:

- Chuyển đổi ảnh màu thành ảnh xám.
- Điều chỉnh độ sáng/độ tương phản của ảnh.
- Lọc ảnh.
- Phát hiện biên cạnh.

2 Cấu trúc thư mục

```
/
├── .vscode
│   ├── c_cpp_properties.json
│   ├── launch.json
│   └── tasks.json
├── bin
│   └── 23122027
├── build
├── images
│   ├── input
│   │   └── test.jpg
│   └── output
├── include
│   ├── imgUtils.hpp
│   └── libs.hpp
├── Makefile
├── run.sh
└── src
    ├── imgUtils.cpp
    └── main.cpp
```

3 Hướng dẫn cài đặt và sử dụng

3.1 Yêu cầu hệ thống và Cài đặt

Đồ án sử dụng ngôn ngữ C++ và thư viện OpenCV để xử lý ảnh. Để biên dịch và chạy chương trình trên môi trường Arch Linux, ta cần cài đặt các công cụ phát triển cơ bản và thư viện tương ứng.

1. **Cập nhật hệ thống và cài đặt các gói cần thiết:** Mở terminal và chạy lệnh sau để cài đặt trình biên dịch (gcc, g++), trình gỡ lỗi (gdb), công cụ build (make), công cụ quản lý thư viện (pkgconf), và thư viện opencv:

```
1 sudo pacman -Syu
2 sudo pacman -S base-devel gdb pkgconf opencv
3
```

2. **Biên dịch chương trình:** Di chuyển terminal vào thư mục gốc của đồ án (nơi chứa file Makefile) và thực thi lệnh make:

```
1 make
2
```

Quá trình này sẽ sử dụng g++ cùng cấu hình từ pkg-config opencv4 để biên dịch mã nguồn trong thư mục src/ và tạo ra file thực thi 23122027 nằm trong thư mục bin/.

3. **Chạy chương trình thủ công:** Sẽ được đề cập ở từng phần mục sau.
4. **Chạy chương trình tự động:** Kịch bản run.sh đã được chuẩn bị sẵn để chạy thử nghiệm tất cả các tính năng của đồ án. Trước khi chạy, đảm bảo bạn đã tạo thư mục images/input/ và đưa một bức ảnh mẫu có tên test.jpg vào đó.

Sau đó, cấp quyền thực thi và chạy script:

```
1 chmod +x run.sh
2 ./run.sh
3
```

Script sẽ tuần tự gọi file thực thi để thực hiện các tác vụ xử lý ảnh và tự động lưu ảnh kết quả vào thư mục images/output/.

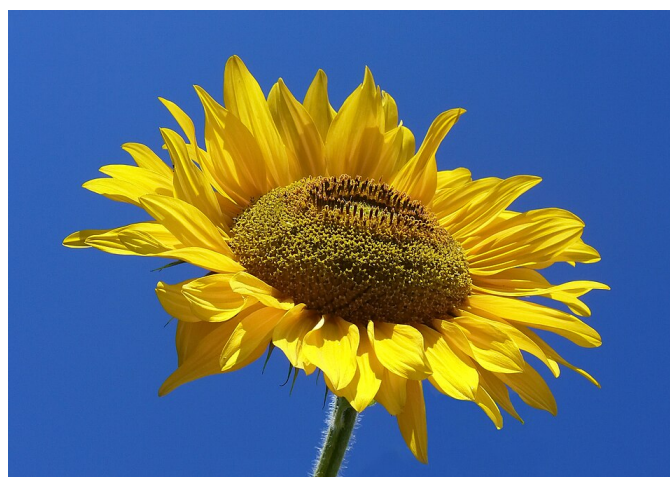
3.2 Cấu hình môi trường Visual Studio Code

Để thuận tiện cho việc phát triển và gỡ lỗi mã nguồn C++, đồ án đã được cấu hình sẵn cho trình soạn thảo Visual Studio Code thông qua các file trong thư mục `.vscode`:

- **Cấu hình IntelliSense (`c_cpp_properties.json`):** File này khai báo chuẩn ngôn ngữ `gnu++17` và cung cấp đường dẫn đến thư viện OpenCV (`/usr/include/opencv4`). Điều này giúp VS Code nhận diện đúng các hàm của OpenCV, hỗ trợ lập trình trên môi trường Linux.
- **Tác vụ biên dịch (`tasks.json`):** Định nghĩa một tác vụ mang tên `build with make`. Tác vụ này tự động thực thi lệnh `make` đã được định nghĩa trong `Makefile`, giúp quá trình biên dịch (build) dự án diễn ra tự động hóa chỉ với một phím tắt.
- **Cấu hình gỡ lỗi (`launch.json`):** Sử dụng trình gỡ lỗi `gdb` của hệ thống. Cấu hình được thiết lập để tự động gọi tác vụ biên dịch trước (`preLaunchTask`), sau đó khởi chạy file thực thi `23122027` với các đối số dòng lệnh mẫu. Thiết lập này cho phép đặt breakpoint và theo dõi giá trị biến một cách trực quan trong quá trình chạy thử nghiệm.

4 Cơ sở lý thuyết của các thuật toán xử lý ảnh

Ta sẽ sử dụng ảnh sau như một ví dụ minh họa cho các thao tác xử lý ảnh:



Hình 1: Ảnh ví dụ (Nguồn: [wikipedia.org](https://en.wikipedia.org/wiki/File:Sunflower.jpg))

4.1 Phép tích chập 2D (2D Convolution) và Xử lý biên (Padding)

Nhiều thuật toán lọc ảnh đều dựa trên phép toán tích chập. Quá trình này sử dụng một ma trận nhỏ gọi là kernel kích thước $k \times k$ trượt dọc theo từng điểm ảnh của ảnh gốc.

Đặc trưng cốt lõi của phép tích chập (để phân biệt với phép tương quan chéo - Cross-correlation) là kernel phải được lật ngược cả hai chiều ngang và dọc trước khi nhân với các điểm ảnh lân cận. Trong mã nguồn của đồ án, thao tác lật này được thực hiện trực tiếp thông qua việc truy xuất chỉ số ngược của mảng kernel.

Xử lý vùng biên (Padding): Khi tâm của kernel nằm ở các điểm ảnh thuộc mép biên của ảnh gốc, một phần của kernel sẽ bị trượt ra bên ngoài ảnh (không có giá trị để tính toán). Để giải quyết vấn đề này, chương trình áp dụng kỹ thuật **Zero Padding**: mở rộng kích thước ảnh gốc bằng cách bao quanh ảnh bởi các viền pixel có giá trị bằng 0. Số lớp viền cần thêm vào được tính bằng $pad = \lfloor k/2 \rfloor$.

Công thức: Giả sử ta có ảnh đầu vào I đã được đệm bước Zero Padding và một kernel K có kích thước $k \times k$ (k là số lẻ). Giá trị của điểm ảnh mới $S(x, y)$ tại tọa độ (x, y) sau khi thực hiện tích chập được tính theo công thức:

$$S(x, y) = \sum_{i=-pad}^{pad} \sum_{j=-pad}^{pad} I(x+i, y+j) \cdot K(-i, -j)$$

Trong đó:

- $S(x, y)$: Giá trị cường độ sáng của điểm ảnh mới trên kết quả.
- $I(x+i, y+j)$: Giá trị cường độ sáng của điểm ảnh trên ảnh gốc tại vùng lân cận xung quanh tọa độ (x, y) .
- $K(-i, -j)$: Trọng số của kernel tại vị trí tương ứng đã được lật ngược theo cả hai chiều. Tâm của kernel được quy ước là tọa độ $(i=0, j=0)$.

4.2 Chuyển đổi ảnh màu sang ảnh xám

Để chuyển đổi ảnh màu RGB (đỏ, xanh lục, xanh lam) sang ảnh mức xám (Grayscale) ta dùng công thức tính giá trị mức xám Y từ các kênh R, G, B được định nghĩa như sau:

$$Y = 0.3 \times R + 0.59 \times G + 0.11 \times B$$

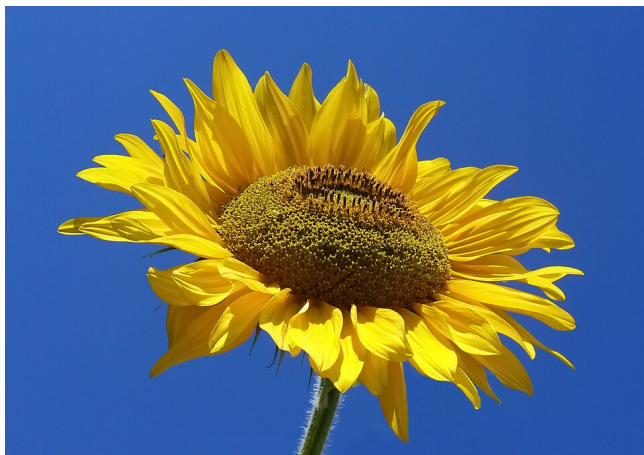
Trong đó:

- Y : Cường độ sáng của điểm ảnh trên ảnh xám kết quả ($0 \leq Y \leq 255$).
- R, G, B : Giá trị cường độ của các kênh màu Đỏ, Xanh lá, Xanh lam của điểm ảnh gốc.
- Các hệ số 0.3, 0.59, 0.11 (tổng bằng 1) chính là các trọng số tương ứng với độ nhạy cảm của mắt người đối với từng dải phổ màu.

Ta sử dụng lệnh:

```
1 ./bin/23122027 -rgb2gray <input_img_path> <output_img_path>
```

Ví dụ minh họa: Sau khi chuyển ảnh màu sang ảnh mức xám ta được



(a) Ảnh gốc



(b) Ảnh kết quả

Hình 2: Kết quả chuyển đổi ảnh màu sang ảnh xám

4.3 Thay đổi độ sáng (Brightness)

Để thay đổi độ sáng của ảnh, ta cộng thêm một hằng số c vào giá trị cường độ của mỗi điểm ảnh. Kết quả sau đó được giới hạn trong khoảng $[0, 255]$ để tránh tràn giá trị.

$$I_{new}(x, y) = I_{old}(x, y) + c$$

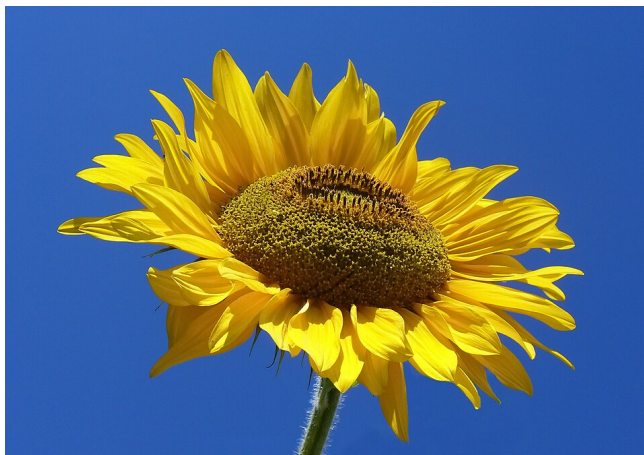
Trong đó:

- $I_{new}(x, y)$: Giá trị cường độ điểm ảnh mới.
- $I_{old}(x, y)$: Giá trị cường độ điểm ảnh ban đầu.
- c : Giá trị thay đổi độ sáng (nếu $c > 0$ ảnh sáng hơn, nếu $c < 0$ ảnh tối đi).

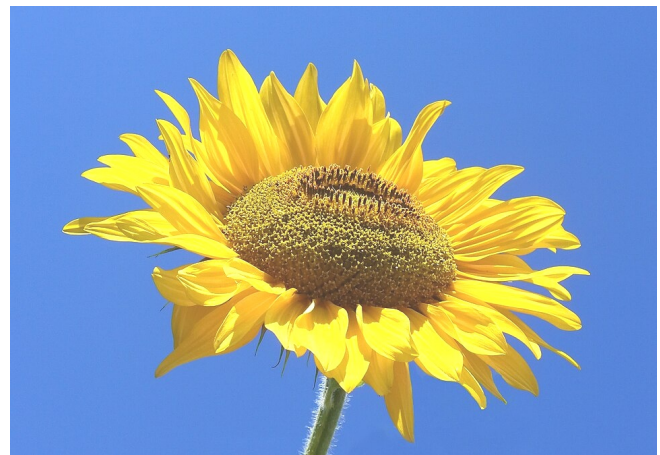
Ta sử dụng lệnh:

```
1 ./bin/23122027 -brightness <input_img_path> <output_img_path> <c>
```

Ví dụ minh họa: Sau khi tăng độ sáng ($c=50$), ta được:



(a) Ảnh gốc



(b) Ảnh kết quả

Hình 3: Ảnh sau khi tăng độ sáng lên 50

4.4 Thay đổi độ tương phản (Contrast)

Để thay đổi độ tương phản, ta nhân giá trị cường độ của mỗi điểm ảnh với một hệ số c . Tương tự như độ sáng, kết quả được giới hạn trong khoảng $[0, 255]$.

$$I_{new}(x, y) = I_{old}(x, y) \times c$$

Trong đó:

- $I_{new}(x, y)$: Giá trị cường độ điểm ảnh mới.
- $I_{old}(x, y)$: Giá trị cường độ điểm ảnh ban đầu.
- c : Hệ số tương phản ($c > 1$: tăng tương phản, $0 < c < 1$: giảm tương phản).

Ta sử dụng lệnh:

```
1 ./bin/23122027 -contrast <input_img_path> <output_img_path> <c>
```

Ví dụ minh họa: Sau khi tăng độ tương phản ($c=1.5$), ta được:



(a) Ảnh gốc



(b) Ảnh kết quả

Hình 4: Ảnh sau khi tăng độ tương phản 1.5 lần

4.5 Bộ lọc trung bình (Average Filter)

Bộ lọc trung bình là một bộ lọc không gian tuyến tính có tác dụng làm trơn ảnh và giảm nhiễu. Phương pháp này hoạt động bằng cách thay thế giá trị cường độ của mỗi điểm ảnh bằng trung bình cộng của các điểm ảnh trong một vùng lân cận kích thước $k \times k$.

Ta dùng một kernel K kích thước $k \times k$, trong đó tất cả các phần tử đều có trọng số bằng nhau và bằng $\frac{1}{k^2}$:

$$K = \frac{1}{k^2} \begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 1 \end{bmatrix}_{k \times k}$$

Trong đó:

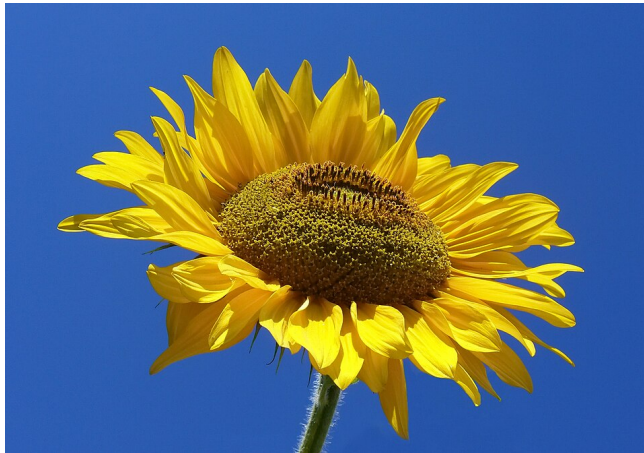
- k : Kích thước của kernel lọc (k là số lẻ nguyên dương, $k > 1$).
- Tổng các phần tử trong kernel K luôn bằng 1 để đảm bảo độ sáng trung bình của bức ảnh không bị thay đổi sau khi lọc.

Ảnh kết quả thu được bằng cách áp dụng phép tích chập 2D giữa ảnh gốc I và kernel K như đã định nghĩa ở phần trước.

Ta sử dụng lệnh:

```
1 ./bin/23122027 -avg <input_img_path> <output_img_path> <k>
```

Ví dụ minh họa: Sau khi áp dụng bộ lọc trung bình ($k = 5$), ta được:



(a) Ảnh gốc



(b) Ảnh kết quả

Hình 5: Ảnh sau khi qua bộ lọc trung bình ($k=5$)

4.6 Bộ lọc trung vị (Median Filter)

Bộ lọc trung vị là một bộ lọc phi tuyến, cực kỳ hiệu quả trong việc loại bỏ nhiễu hạt (như nhiễu muối tiêu - salt and pepper) mà vẫn giữ được độ sắc nét của biên ảnh. Giá trị điểm ảnh được thay thế bằng giá trị trung vị của các điểm ảnh trong vùng lân cận $k \times k$.

$$I_{new}(x, y) = \text{median}\{I_{old}(x + i, y + j) \mid i, j \in [-pad, pad]\}$$

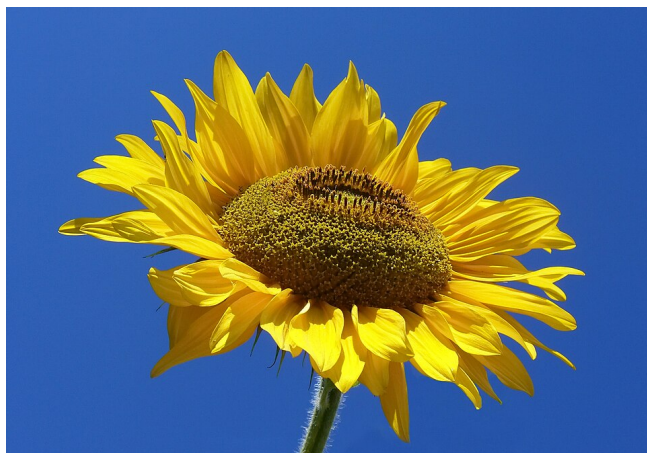
Trong đó:

- pad : Bán kính lân cận, với $pad = \lfloor k/2 \rfloor$.
- k : Kích thước của sổ lọc (k là số lẻ dương > 1).

Ta sử dụng lệnh:

```
1 ./bin/23122027 -med <input_img_path> <output_img_path> <k>
```

Ví dụ minh họa: Sau khi áp dụng bộ lọc trung vị ($k = 5$), ta được:



(a) Ảnh gốc



(b) Ảnh kết quả

Hình 6: Ảnh sau khi qua bộ lọc trung vị ($k=5$)

4.7 Bộ lọc Gauss (Gaussian Filter)

Bộ lọc Gauss làm mờ ảnh bằng cách sử dụng một kernel tích chập mà trọng số của các phần tử giảm dần từ tâm ra biên theo phân bố chuẩn 2D.

Trong đề án này, để tự động hóa, độ lệch chuẩn σ được tính toán trực tiếp dựa trên kích thước kernel k (với k lẻ) theo công thức:

$$\sigma = 0.3 \times \left(\frac{k-1}{2} - 1 \right) + 0.8$$

Trọng số ban đầu tại vị trí (i, j) so với tâm của kernel (với $i, j \in [-pad, pad]$ và $pad = \lfloor k/2 \rfloor$) được tính bằng hàm phân bố Gauss:

$$G(i, j) = \frac{1}{2\pi\sigma^2} e^{-\frac{i^2+j^2}{2\sigma^2}}$$

Chuẩn hóa kernel (Normalization): Để đảm bảo tổng các trọng số trong kernel bằng 1 (tránh làm thay đổi độ sáng tổng thể của ảnh), mỗi phần tử trong kernel sẽ được chia cho tổng của tất cả các phần tử trước khi thực hiện phép tích chập:

$$K(i, j) = \frac{G(i, j)}{\sum_{u=-pad}^{pad} \sum_{v=-pad}^{pad} G(u, v)}$$

Ta sử dụng lệnh:

```
1 ./bin/23122027 -gau <input_img_path> <output_img_path> <k>
```


Ví dụ minh họa: Sau khi áp dụng bộ lọc Gauss ($k = 5$), ta được:



(a) Ảnh gốc



(b) Ảnh kết quả

Hình 7: Ảnh sau khi qua bộ lọc Gauss ($k=5$)

4.8 Phát hiện biên cạnh bằng Sobel kernel (Sobel Edge)

Toán tử Sobel phát hiện biên ảnh bằng cách tính xấp xỉ đạo hàm bậc nhất của cường độ sáng. Trước tiên, ảnh màu (nếu có) sẽ được chuyển sang ảnh mức xám. Sau đó, ảnh mức xám được tích chập lần lượt với hai kernel 3×3 để tìm gradient theo chiều ngang (x) và chiều dọc (y):

kernel gradient theo chiều ngang (K_x) và chiều dọc (K_y):

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Giả sử kết quả tích chập của ảnh gốc với K_x và K_y lần lượt là G_x và G_y . Độ lớn gradient tại mỗi điểm ảnh được tính bằng căn bậc hai tổng bình phương của hai thành phần:

$$G = \sqrt{G_x^2 + G_y^2}$$

Kết quả cuối cùng G sẽ được giới hạn về khoảng số nguyên $[0, 255]$ để hiển thị.

Ta sử dụng lệnh:

```
1 ./bin/23122027 -sobel <input_img_path> <output_img_path>
```

Ví dụ minh họa: Sau khi phát hiện biên bằng Sobel, ta được:



(a) Ảnh gốc



(b) Ảnh kết quả

Hình 8: Kết quả phát hiện biên Sobel

4.9 Phát hiện biên cạnh bằng Laplace kernel (Laplace Edge)

Toán tử Laplace sử dụng đạo hàm bậc hai để phát hiện biên, cho phép tìm biên trên mọi hướng (đẳng hướng) chỉ với một lần tích chập. Tương tự Sobel, ảnh đầu vào sẽ được chuyển sang mức xám trước khi xử lý.

Đồ án sử dụng ma trận kernel Laplace như sau:

$$K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

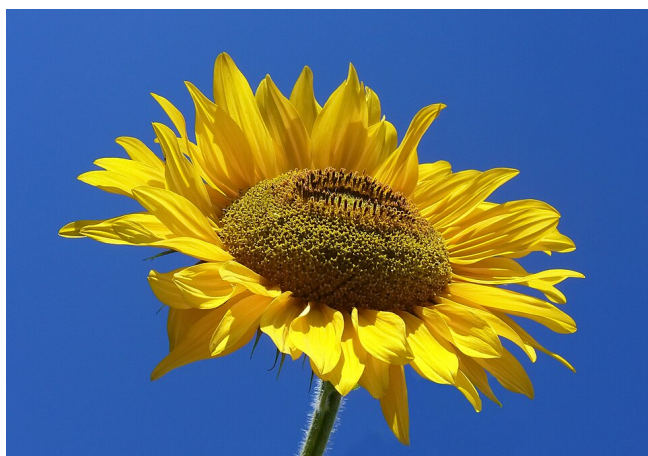
Đặc điểm của đạo hàm bậc hai là nó tạo ra các giá trị âm tại bề mặt chuyển từ sáng sang tối và giá trị dương tại bề mặt chuyển từ tối sang sáng. Do đó, sau khi áp dụng phép tích chập với ma trận K , chương trình thực hiện **lấy giá trị tuyệt đối** của toàn bộ các điểm ảnh để giữ lại thông tin biên ở cả hai chiều biến thiên, trước khi giới hạn về chuẩn 8-bit để hiển thị:

$$I_{new} = |I_{old} * K|$$

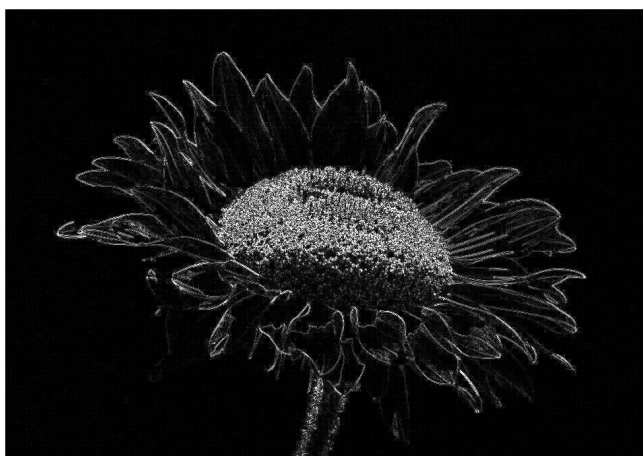
Ta sử dụng lệnh:

```
1 ./bin/23122027 -laplace <input_img_path> <output_img_path>
```

Ví dụ minh họa: Sau khi phát hiện biên bằng Laplace, ta được:



(a) Ảnh gốc



(b) Ảnh kết quả

Hình 9: Kết quả phát hiện biên Laplace